GREEDY METHODS

A greedy algorithm:

- is any algorithm that follows the problem solving *metaheuristic* of making the locally optimal choice at each stage with the hope of finding the global optimum.

A metaheuristic method:

- Is method for solving a very general class of computational problems that aims on obtaining a more efficient or more robust procedure for the problem.
- Generally it is applied to problems for which there is no satisfactory problem-specific algorithm designed to solve it.
- It targeted to the *combinatorial optimization* (problems in which has an optimization function to(minimize or maximize) subject to some constraints and its goal is to find the best possible solution

EXAMPLES

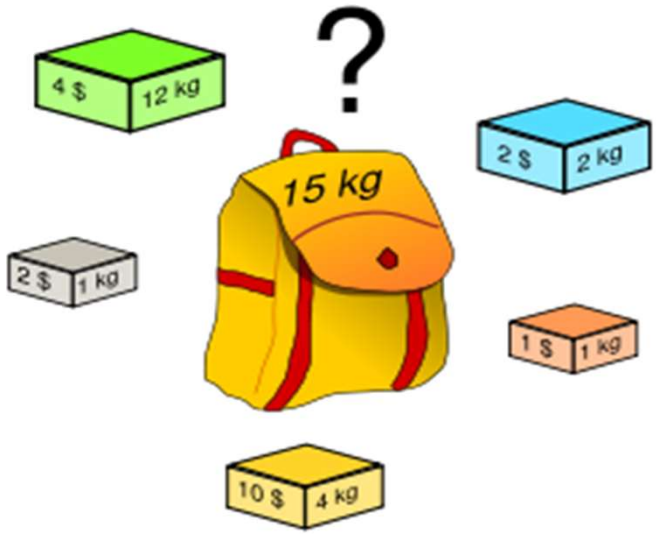
- The vehicle routing problem (VRP)
  - A number of goods need to be moved from certain pickup locations to other delivery locations. The goal is to find optimal routes for a fleet of vehicles to visit the pickup and drop-off locations.
- Travelling salesman problem
  - Given a list of cities and their pair wise distances, the task is to find a shortest possible tour that visits each city exactly once.
- Coin Change
  - (making change for n $ using minimum number of coins)
- The knapsack problem
- The Shortest Path Problem

KNAPSACK

- The **knapsack problem** or **rucksack problem** is a problem in combinatorial optimization. It derives its name from the following maximization problem of the best choice of essentials that can fit into one bag to be carried on a trip. Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than a given limit and the total value is as large as possible.

The Original Knapsack Problem (1)

- Problem Definition
  - Want to carry essential items in one bag
  - Given a set of items, each has
    - A Weight (i.e., 12kg)
    - A value (i.e., 4$)



- Goal
  - To determine the # of each item to include in a collection so that
    - The total cost is less than some given cost
    - And the total value is as large as possible

The Original Knapsack Problem (2)

- Three Types
  - 0/1 Knapsack Problem
    - restricts the number of each kind of item to zero or one
  - Bounded Knapsack Problem
    - restricts the number of each item to a specific value
  - Unbounded Knapsack Problem
    - places no bounds on the number of each item

0/1 Knapsack Problem (1)

- Problem: Person A wishes to take n items on a trip
  - The weight of item i is w_i & items are all different (0/1 Knapsack Problem)
  - The items are to be carried in a knapsack whose weight capacity is c
    - When sum of item weights ≤ c, all n items can be carried in the knapsack
    - When sum of item weights > c, some items must be left behind
- Which items should be taken/left?

0/1 KNAPSACK PROBLEM (2)

- Person A assigns a profit p_i to item i
  - All weights and profits are positive numbers
- Person A wants to select a subset of the n items to take
  - The weight of the subset should not exceed the capacity of the knapsack (constraint)
  - Cannot select a fraction of an item (constraint)
  - The profit of the subset is the sum of the profits of the selected items (optimization function)
  - The profit of the selected subset should be maximum (optimization criterion)
- Let $x_i = 1$ when item i is selected and $x_i = 0$ when item i is not selected
  - Because this is a 0/1 Knapsack Problem, you can choose the item or not choose it.

1 0/1 Knapsack

Given a set of n objects, say $x_1, x_2, \ldots, x_n$, along with its weight $w_1, w_2, \ldots, w_n$ and profits $p_1, p_2, \ldots, p_n$. The objective is to find $S \subseteq \{x_1, x_2, \ldots, x_n\}$ such that $Profit(S) = \sum_{x_i \in S} p_i$ is maximum subject to the constraint $\sum_{x_i \in S} w_i \leq W$, where W is the capacity of the knapsack. This problem is called as 0/1 knapsack because for all x_i, either $x_i = 1$ (i.e., the object x_i is included in the knapsack) or $x_i = 0$ (i.e., the object x_i is not included in the knapsack). Recall that this problem can be solved using the dynamic programming paradigm in pseudo-polynomial time ($O(nW)$). Note that if $W = 2^n$, then the algorithm runs in exponential in n.

Greedy with respect to weight:

Step 1: Sort the weights in increasing order, say $w_1 \leq w_2 \leq \ldots \leq w_n$. This step takes $O(n \log n)$ time.

Step 2: Pick the minimum weight (local optimum), which is w_1 in this case. Check $w_1 \leq W$. If so, add x_1 to S.

Step 3: Check $w_1 + w_2 \leq W$. If so, add x_1 and x_2 to S.

Step 4: Proceed this process until we get the least j such that $w_1 + w_2 + \ldots + w_j > W$. Now return $S = \{x_1, \ldots, x_{j-1}\}$.

The Steps 2-4 take $O(n)$. Overall, the above approach takes $O(n \log n)$ time.

The solution obtained by this greedy approach is a feasible solution, since, $\sum_{x_i \in S} w_i \leq W$.

Query: Is the solution obtained by greedy w.r.t weight is optimum ?

Solution: No. Here is a counter example:

The sorted weights and the corresponding profits are as follows

Given the weight of the knapsack is 5, our algorithm picks the first two objects as an output (Since,

Objects	x_1	x_2	x_3	x_4
Weights	1	2	4	5
Profits	2	2	3	5

$w_1 + w_2 + w_3 = 1+2+4 = 7 > W$ and $w_1 + w_2 = 1+2 = 3 \leq W$) and the corresponding profit is $2+2 = 4$.

Greedy with respect to profit:

Step 1: Sort the weights in decreasing order, say $p_1 \geq p_2 \geq \ldots \geq p_n$. Let $w_1, w_2, \ldots, w_n$ be the corresponding weights for $p_1 \geq p_2 \geq \ldots \geq p_n$.

Step 2: Pick the maximum profit (local optimum), which is p_1 in this case. Check $w_1 \leq W$. If so, add x_1 to S.

Step 3: Check $w_1 + w_2 \leq W$. If so, add x_1 and x_2 to S.

Step 4: Proceed this process until we get the least j such that $w_1 + w_2 + \ldots + w_j > W$. Now return $S = \{x_1, \ldots, x_{j-1}\}$.

The solution obtained by this greedy approach is a feasible solution, since, $\sum_{x_i \in S} w_i \leq W$.

Query: Is the solution obtained by greedy w.r.t profit is optimum ?
Solution: No. Here is a counter example:
The sorted profits and the corresponding weights are as follows

Objects	x_1	x_2	x_3	x_4
Profits	4	3	2	1
Weights	5	1	4	5

Given the weight of the knapsack is 5, our algorithm outputs $S = \{x_1\}$ (Since, $w_1 + w_2 = 5 + 1 = 6 > W$ and $w_1 = 5 \leq W$) and the corresponding profit is 4. The optimum solution is $\{x_2, x_3\}$ and the profit is 5. Thus, greedy with respect to profit does not give optimum solution always.

Both the greedy with respect to weight and the greedy with respect to profit fail, because in some cases the objects chosen by the algorithm gives less profit than any optimum. So, now let us try yet another greedy approach with respect to profit per unit weight(profit/weight).

Greedy with respect to profit/weight:

Step 1: Sort the weights in decreasing order, say $p_1/w_1 \geq p_2/w_2 \geq \ldots \geq p_n/w_n$.

Step 2: Pick the maximum p_i/w_i (local optimum), which is p_1/w_1 in this case. Check the corresponding $w_1 \leq W$. If so, add x_1 to S.

Step 3: Check $w_1 + w_2 \leq W$. If so, add x_1 and x_2 to S.

Step 4: Proceed this process until we get the least j such that $w_1 + w_2 + \ldots + w_j > W$. Now return $S = \{x_1, \ldots, x_{j-1}\}$.

The solution obtained by this greedy approach is a feasible solution, since, $\sum_{x_i \in S} w_i \leq W$.

Query: Is the solution obtained by greedy w.r.t profit/weight is optimum ?

Solution: No. Here is a counter example:

The sorted weights and the corresponding profits are as follows

Objects	x_1	x_2	x_3
Profits	7	9	6
Weights	3	5	4
Profit/weight	2.33	1.8	1.5

Fractional Knapsack

Given a set of n objects, say $x_1, x_2, \ldots, x_n$, along with its weight $w_1, w_2, \ldots, w_n$ and profits $p_1, p_2, \ldots, p_n$. The objective is to find a subset $S \subseteq \{fx_1, fx_2, \ldots, fx_n\}$, such that $Profit(S) = \sum_{x_i \in S} fx_i \cdot p_i$ is maximum subject to the constraint $\sum_{x_i \in S} w_i \leq W$, where fx_i is the fraction of x_i and $0 \leq fx_i \leq 1$ (so far, we are allowed to include x_i or to exclude x_i, now, we can choose a fraction of x_i which varies between 0 and 1), and W is the capacity of the knapsack. Now let us try to establish a greedy approach for the given problem as follows and we will also check whether the solution obtained by the greedy is optimum or not.

Greedy Strategy 1: with respect to weight:

Step 1: Sort the weights in increasing order, say $w_1 \leq w_2 \leq \ldots \leq w_n$.

Step 2: Pick the minimum weight (local optimum), which is w_1 in this case. Check $w_1 \leq W$. If so, add x_1 to S.

Step 3: Check $w_1 + w_2 \leq W$. If so, add x_1 and x_2 to S.

Step 4: Proceed this process until we get the least j such that $w_1 + w_2 + \ldots + w_j > W$. With respect to w_j, calculate $fx_j = \dfrac{W - \sum_{i=1}^{j-1} w_i}{w_j}$. Now return $S = \{x_1, \ldots, x_{j-1}, fx_j\}$.

The solution obtained by this greedy approach is a feasible solution, since, $\sum_{x_i \in S} (fx_i \cdot w_i) \leq W$.

THE SHORTEST PATH PROBLEM

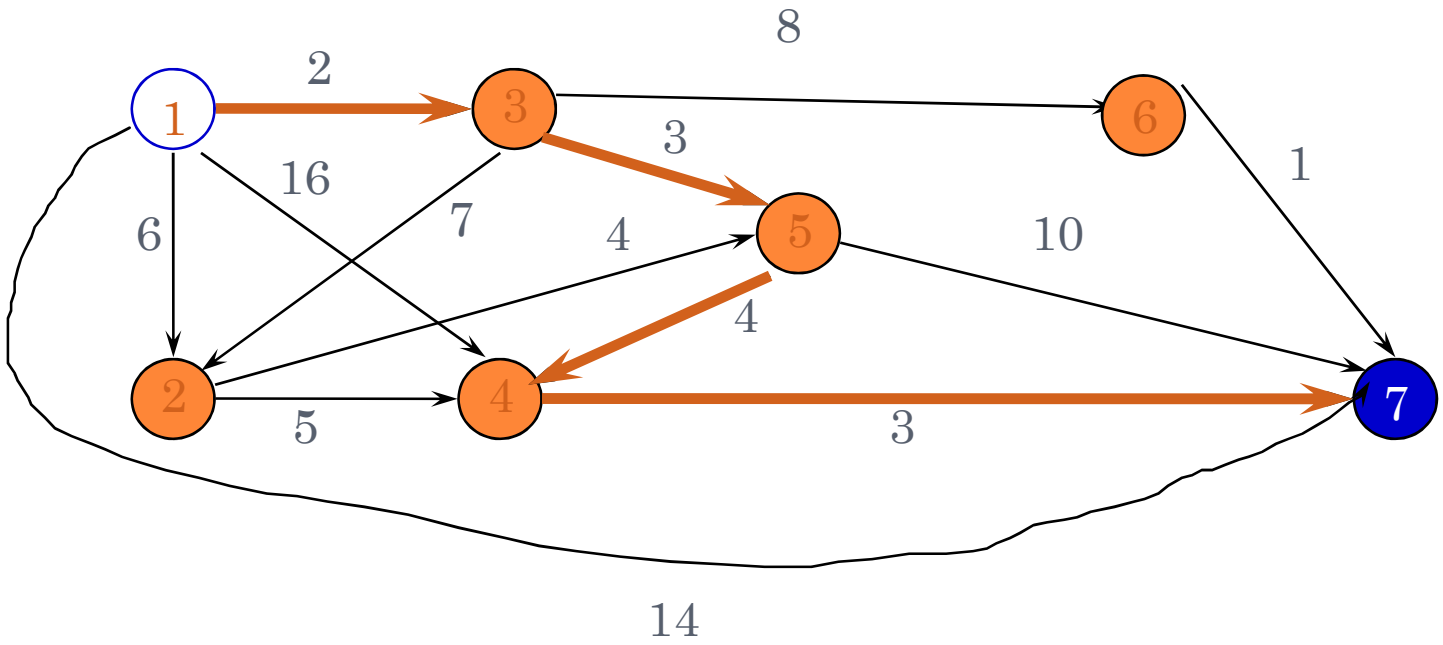
- Path length is sum of weights of edges on path in directed weighted graph
  - The vertex at which the path begins is the source vertex
  - The vertex at which the path ends is the destination vertex
- Goal
  - To find a path between two vertices such that the sum of the weights of its edges is minimized

Types of The Shortest Path Problem

- Three types
  - Single-source single-destination shortest path
  - Single-source all-destinations shortest path
  - All pairs (every vertex is a source and destination) shortest path

SINGLE-SOURCE SINGLE-DESTINATION SHORTED PATH

- Possible greedy algorithm
  - Leave the source vertex using the cheapest edge
  - Leave the current vertex using the cheapest edge to the next vertex
  - Continue until destination is reached
- Try Shortest 1 to 7 Path by this Greedy Algorithm
  - the algorithm does not guarantee the optimal solution



Greedy Single-Source All-Destinations Shortest Path (1)

- Problem: Generating the shortest paths in increasing order of length from one source to multiple destinations

- Greedy Solution
  - Given n vertices, First shortest path is from the source vertex to itself
    - The length of this path is 0
  - Generate up to n paths (including path from source to itself) by the greedy criteria
    - from the vertices to which a shortest path has not been generated, select one that results in the least path length
    - Construct up to n paths in order of increasing length

Note:

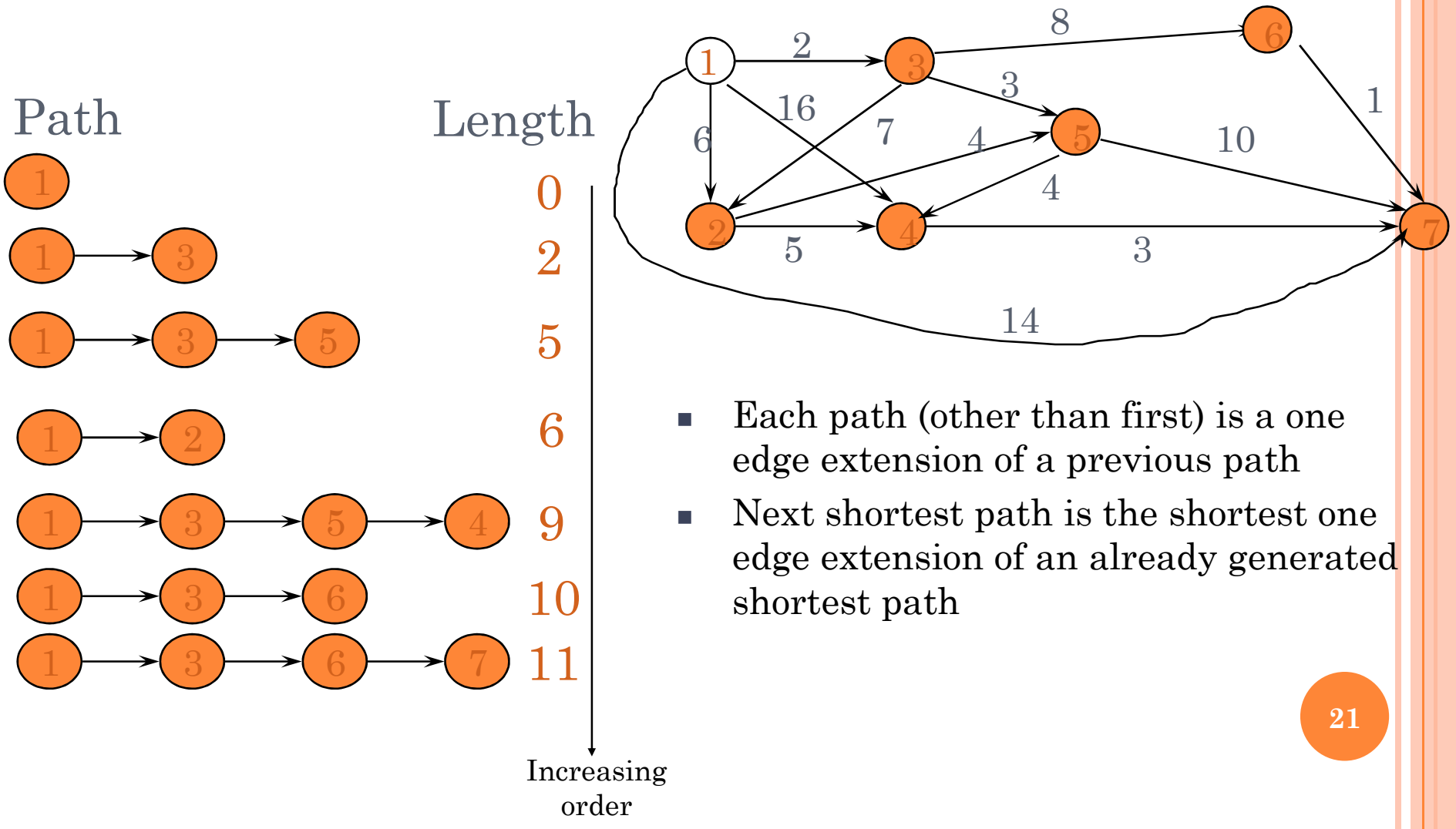
The solution to the problem consists of up to n paths.

The greedy method suggests building these n paths in order of increasing length.

First build the shortest of the up to n paths (i.e., the path to the nearest destination).

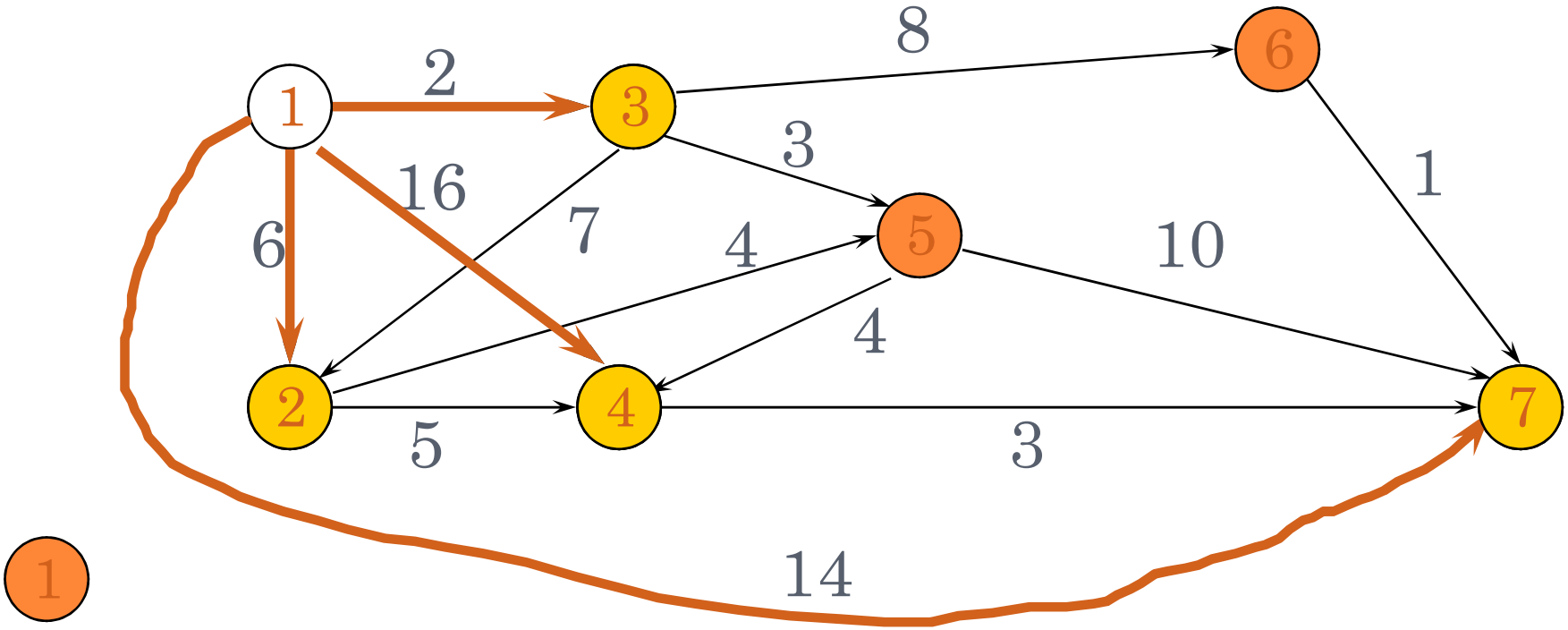
Then build the second shortest path, and so on.

Greedy Single-Source All-Destinations Shortest Path (2)



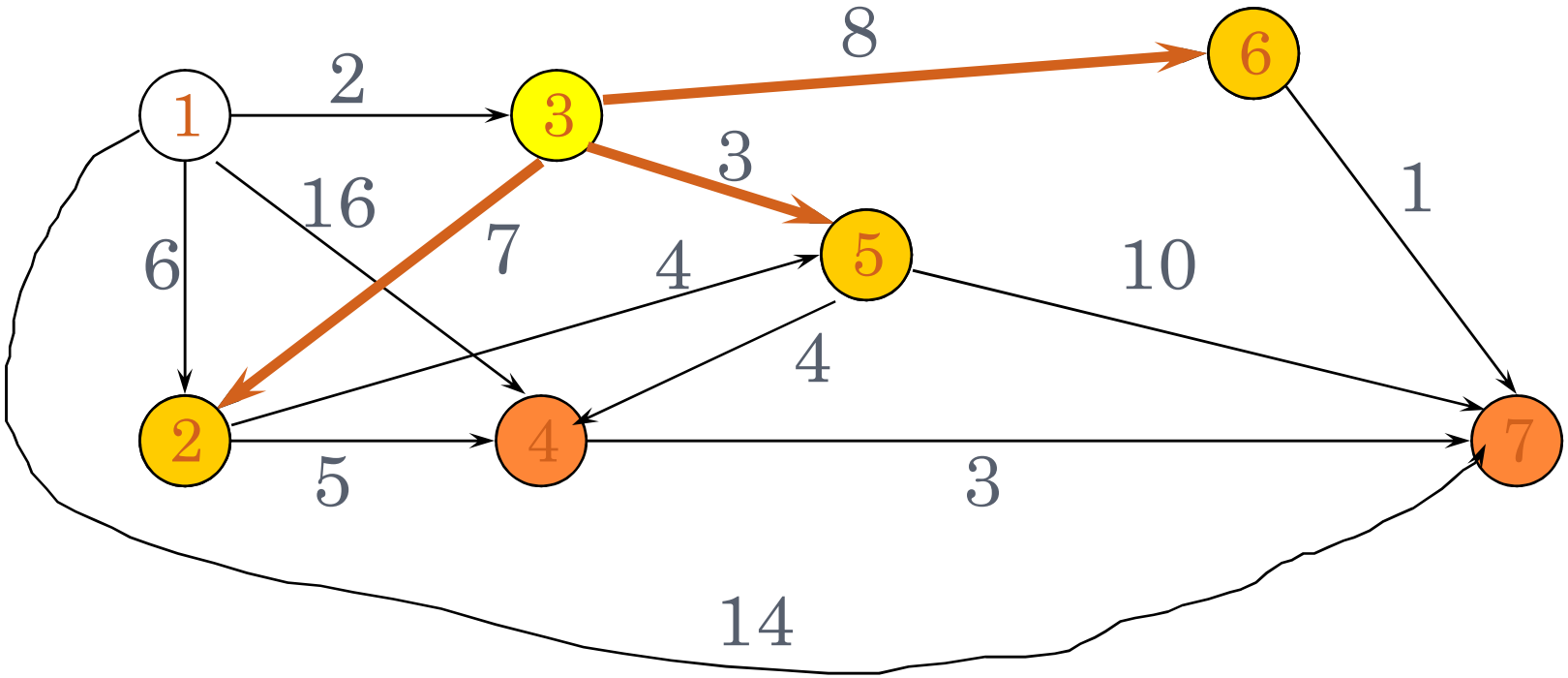
- Each path (other than first) is a one edge extension of a previous path
- Next shortest path is the shortest one edge extension of an already generated shortest path

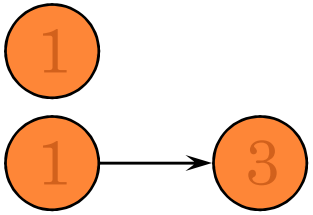
GREEDY SINGLE SOURCE ALL DESTINATIONS: EXAMPLE (1)



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	16	-	-	14
p	-	1	1	1	-	-	1

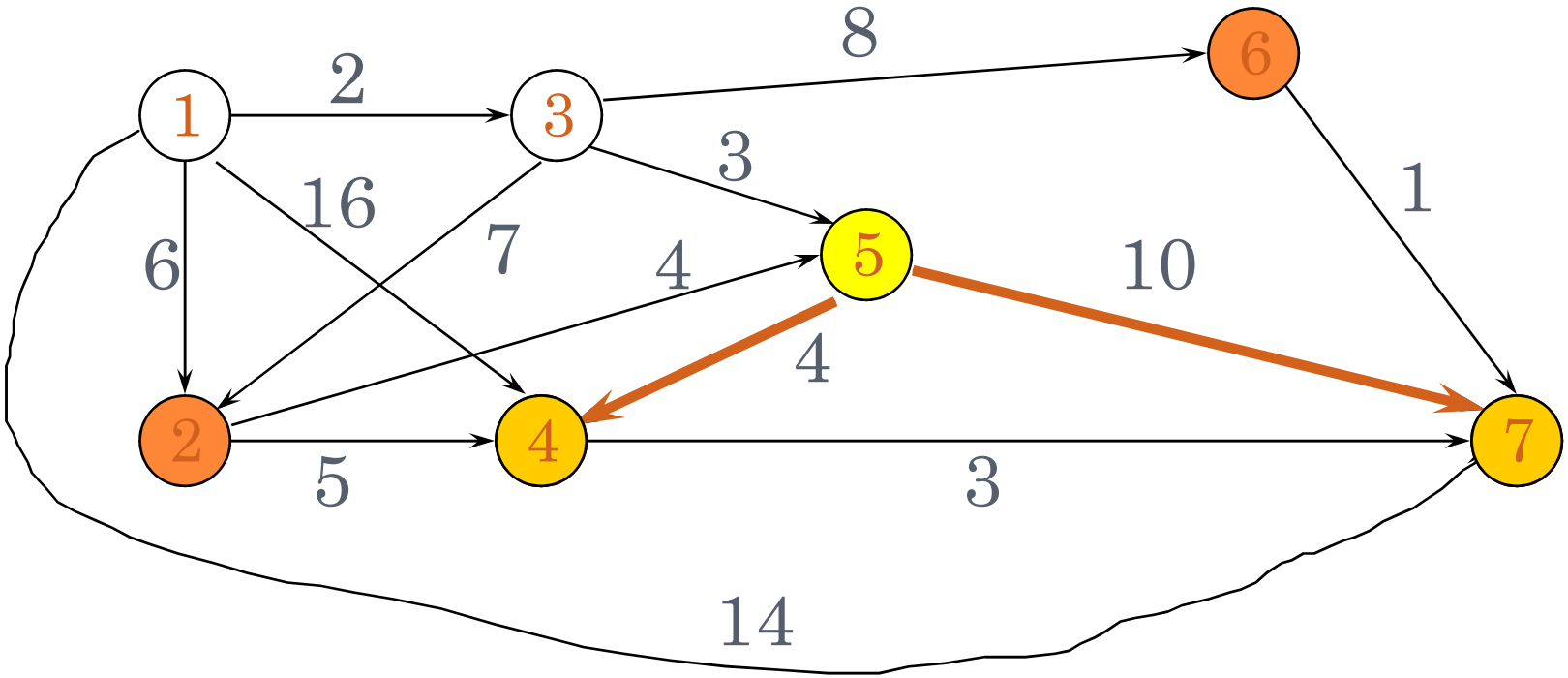
Greedy Single Source All Destinations : Example (2)

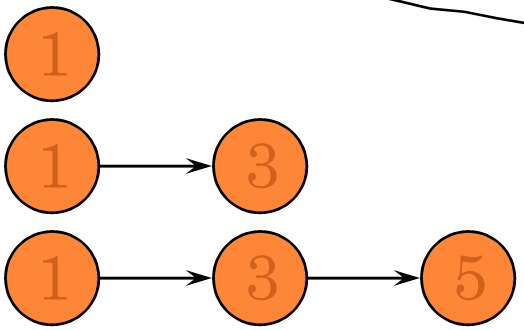




	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	16	5	10	14
p	-	1	1	1	3	3	1

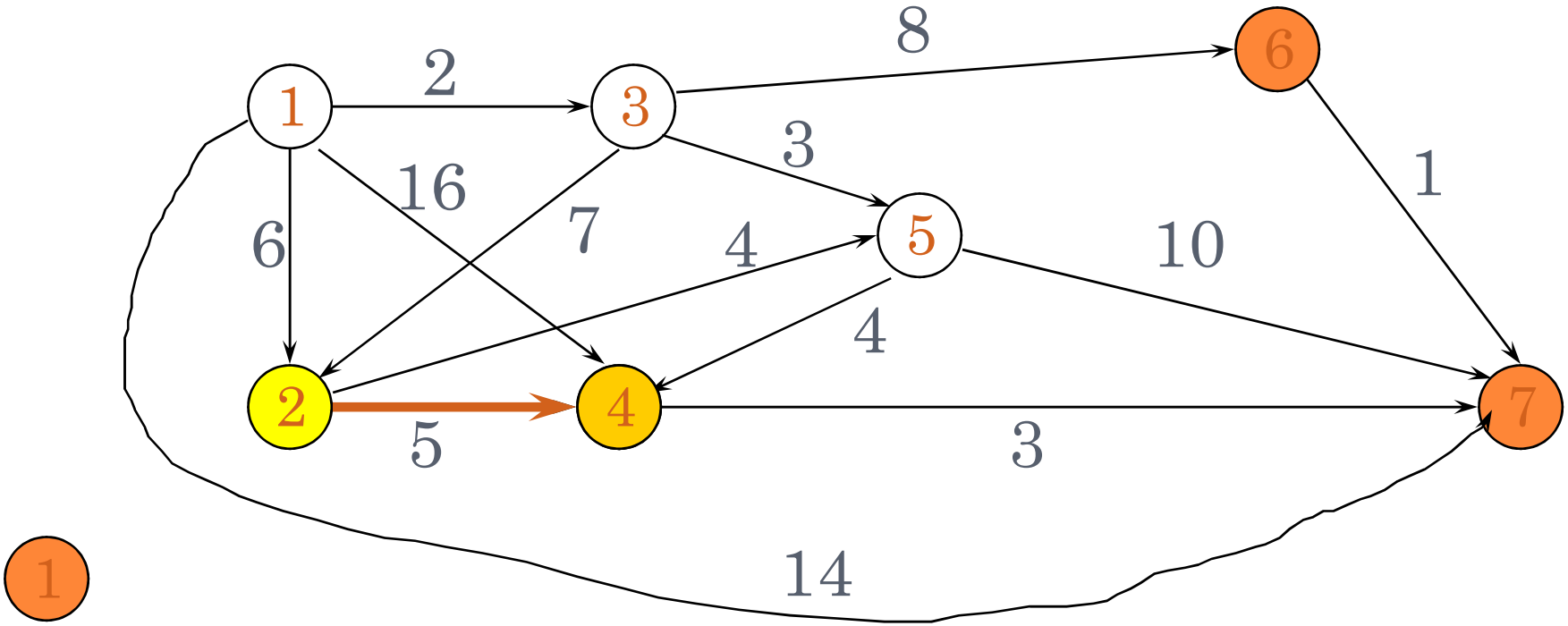
Greedy Single Source All Destinations : Example (3)

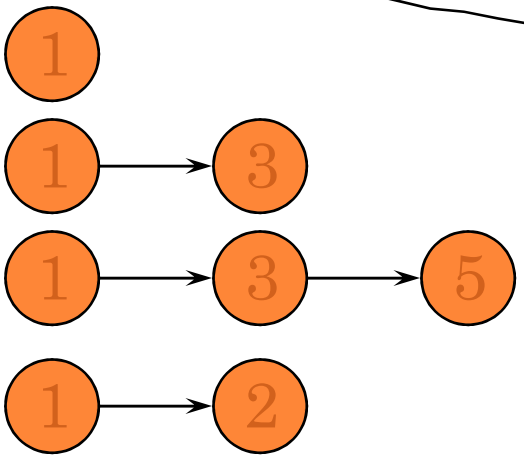




	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	14
p	-	1	1	5	3	3	1

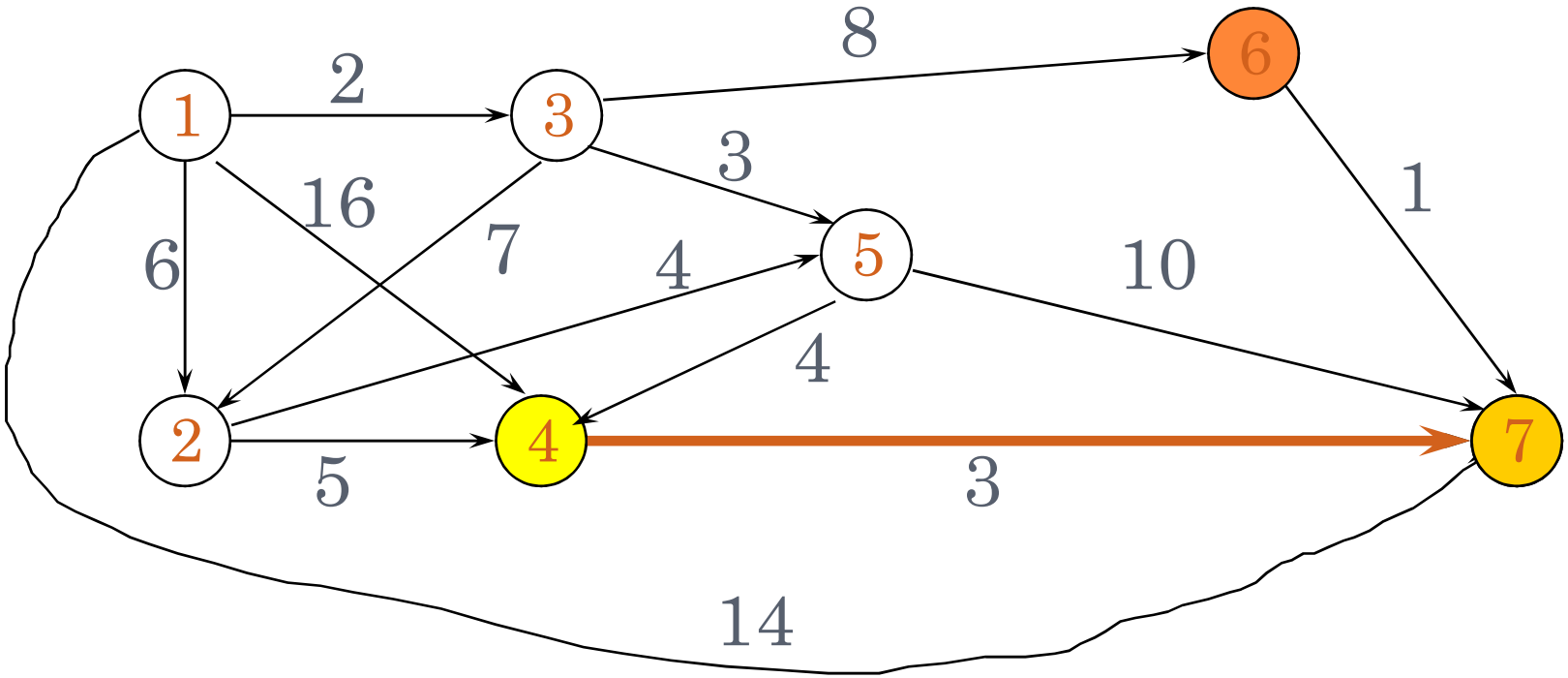
Greedy Single Source All Destinations : Example (4)

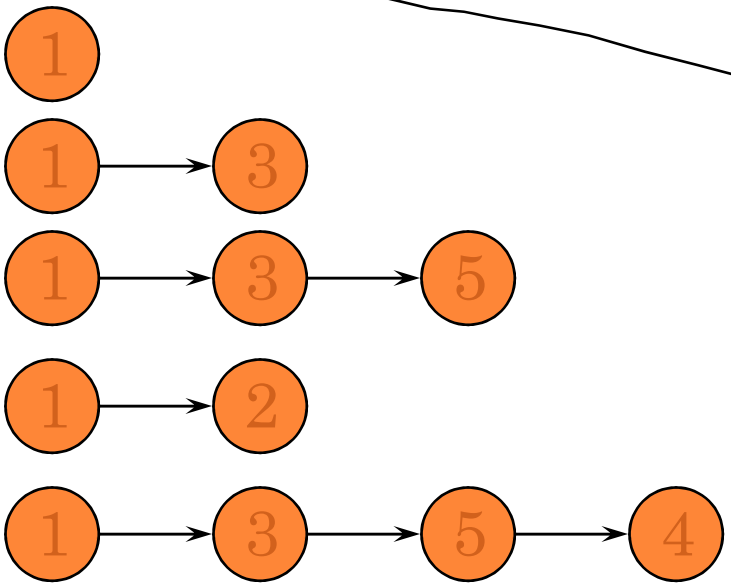




	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	14
p	-	1	1	5	3	3	1

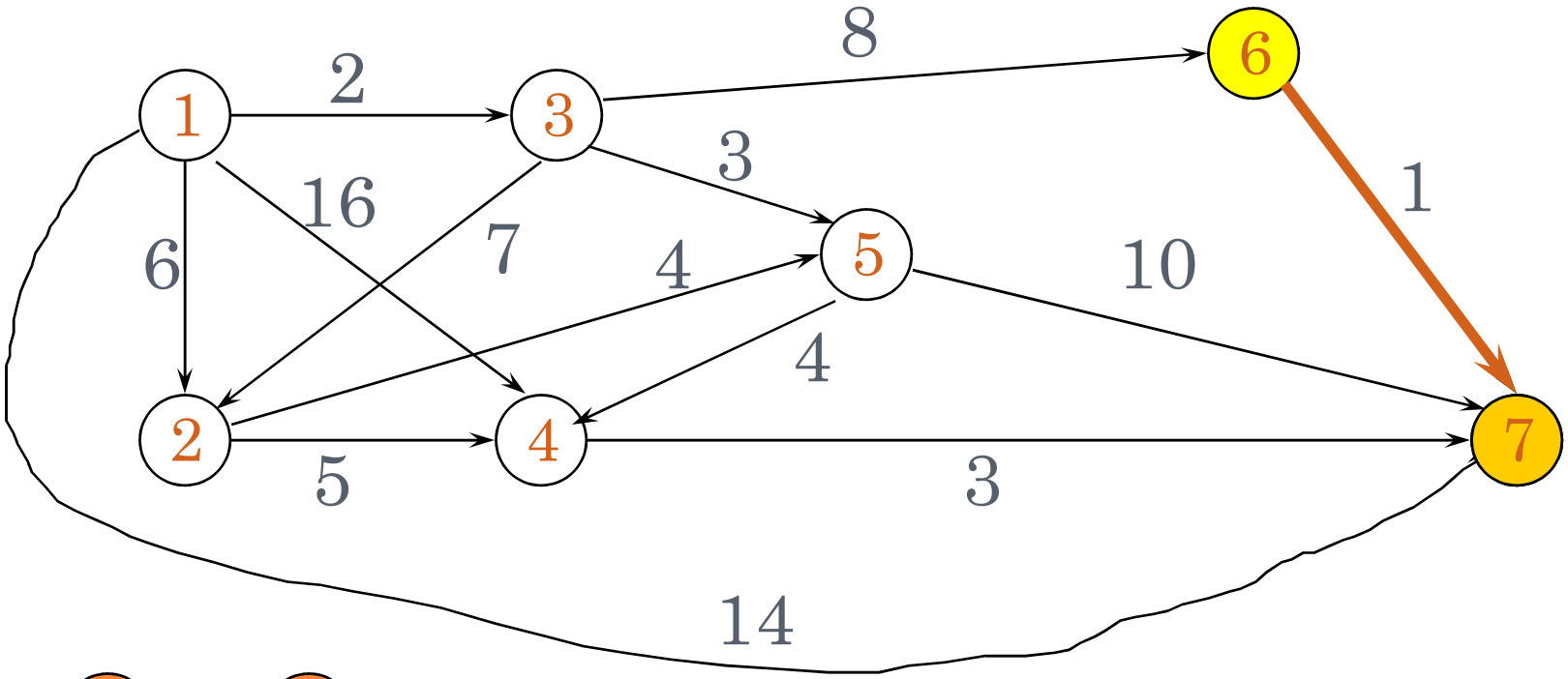
Greedy Single Source All Destinations : Example (5)





	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	1	12
p	-	1	1	5	3	03	4

Greedy Single Source All Destinations : Example (6)



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	11
p	-	1	1	5	3	3	6

Dynamic Programming

Dynamic programming comes from the idea of storing the already calculated states. When calculating the answer for a state, first, we check to see if we have calculated the same state before. If we have already calculated this state, we can just return its answer. Otherwise, we must calculate the answer for this state and store it.

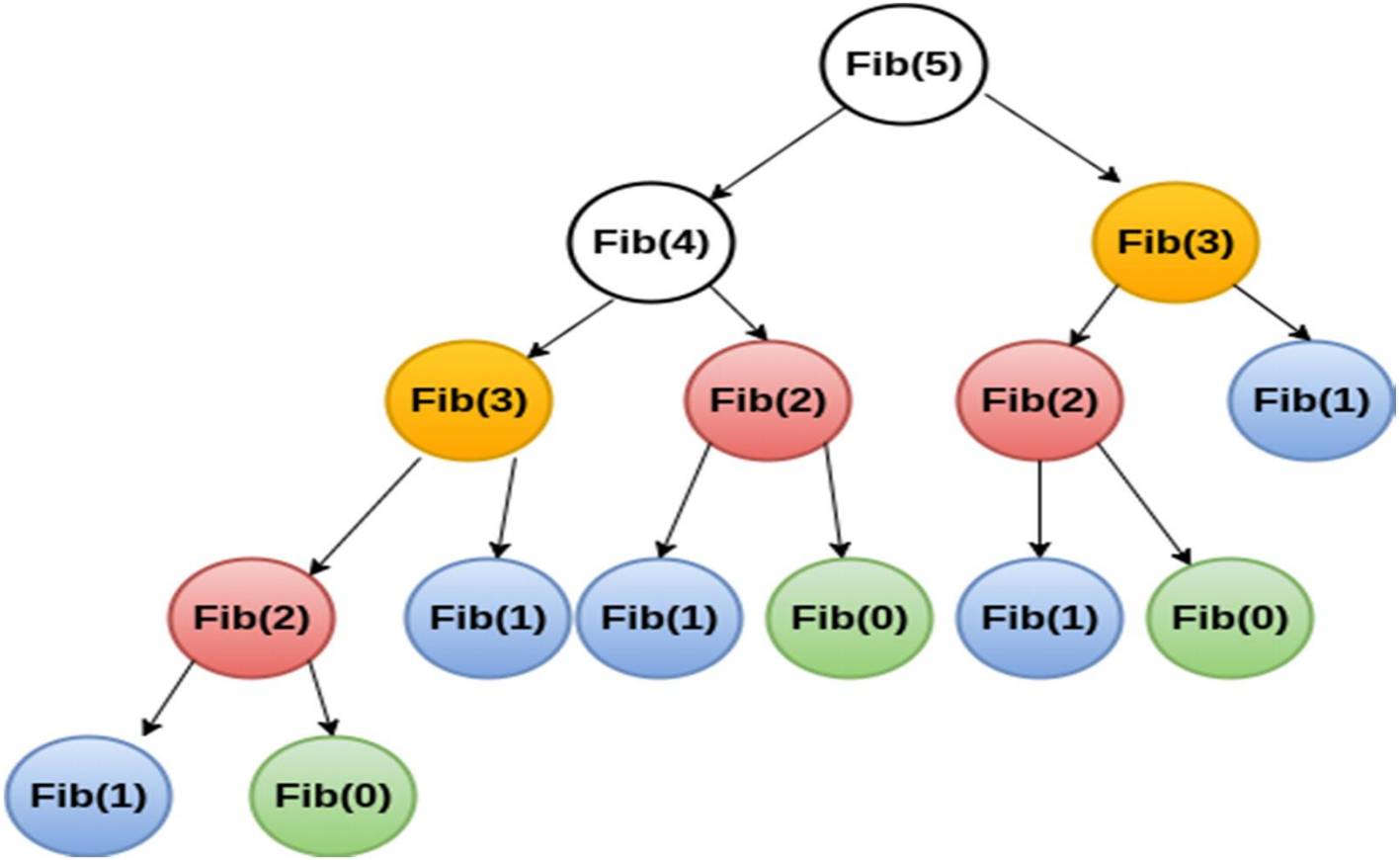
This approach is called top-down dynamic programming. However, there's another approach called the bottom-up approach which mainly has the same idea.

The difference is that the bottom-up approach starts from small subproblems and tries to build up the answer for bigger subproblems. This process continues until we reach the solution to the full problem.

The main benefit of using dynamic programming is that we move to polynomial time complexity, instead of the exponential time complexity in the backtracking version. Also, dynamic programming, if implemented correctly, guarantees that we get an optimal solution.

The reason behind dynamic programming optimality is that **it's an optimization over the backtracking approach which explores all the possible choices.**

Overlapping sub-problems - Example



Greedy vs. Dynamic Programming

- The knapsack problem is a good example of the difference.

- ***0-1 knapsack problem:*** not solvable by greedy.
  - n items.
  - Item i is worth $\$v_i$, weighs w_i pounds.
  - Find a most valuable subset of items with total weight $\leq W$.
  - Have to either take an item or not take it—can't take part of it.

- ***Fractional knapsack problem:*** solvable by **greedy**
  - Like the 0-1 knapsack problem, but can take fraction of an item.
  - Both have optimal substructure.
  - But the fractional knapsack problem has the greedy-choice property, and the 0-1 knapsack problem does not.
  - To solve the fractional problem, rank items by value/weight: $\boldsymbol{v_i / w_i}$.
  - Let $v_i / w_i \geq v_{i+1} / w_{i+1}$ for all i.

The greedy method cannot always find an optimal solution!